SOFTWARE REQUIREMENTS SPECIFICATION (SRS)

DocuQuery AI: Enterprise Retrieval-Augmented Generation Platform

Document Version: 1.0.0
Date: October 2026
Course: Java Project Based Learning (PBL) — 5th Semester, Dept. of AI&DS (Artificial Intelligence & Data Science)
Institution: Arya College of Engineering & I.T., Jaipur
Mentor: Er. Ram Babu Buri (Assistant Professor)
Team Members:

1. Shaurya Dixit (Team Leader & Core RAG Architect)
2. Yashvardhan Singh Chauhan (Data Pipeline Engineer & Kaggle Ingestion Lead)
3. Vaibhav Tank (Database & Security Architect)
4. Rajveer Singh Ranawat (Frontend & Desktop Lead)
5. Yashpal Singh Bhati (QA & Evaluation Lead)

Data Corpus: Kaggle Academic Research Paper Dataset
Standard Compliance: IEEE Std 830-1998

TABLE OF CONTENTS

1. Introduction
   - 1.1 Purpose
   - 1.2 Scope
   - 1.3 Definitions, Acronyms, and Abbreviations
   - 1.4 References
   - 1.5 Overview
2. Overall Description
   - 2.1 Product Perspective
   - 2.2 Product Functions
   - 2.3 User Classes and Characteristics
   - 2.4 Operating Environment
   - 2.5 Design and Implementation Constraints
   - 2.6 Assumptions and Dependencies
3. System Features & Functional Requirements
4. External Interface Requirements
5. Non-Functional Requirements (NFR)
6. Other Requirements & Compliance

---

1. INTRODUCTION

1.1 Purpose

This Software Requirements Specification (SRS) establishes the complete functional and non-functional requirements for the **DocuQuery AI** system. It serves as the baseline contract between the development team and academic evaluators for verification, validation, and grading.

1.2 Scope

DocuQuery AI is a cross-platform desktop and web enterprise application designed to eliminate Large Language Model (LLM) hallucinations through Retrieval-Augmented Generation (RAG). The system accepts unstructured academic and enterprise text corpora (PDF, DOCX, TXT, MD), generates dense vector embeddings, indexes them into a serverless PostgreSQL database using `pgvector` HNSW indexing, and streams grounded, citation-verified natural language answers to users.

1.3 Definitions, Acronyms, and Abbreviations

- **RAG**: Retrieval-Augmented Generation
- **LLM**: Large Language Model
- **HNSW**: Hierarchical Navigable Small World (graph-based vector search)
- **Cosine Similarity**: Metric evaluating angular distance between normalized vector spaces
- **JWT**: JSON Web Token
- **RBAC**: Role-Based Access Control
- **SSE**: Server-Sent Events
- **IPC**: Inter-Process Communication (Electron)

1.4 References

- IEEE Std 830-1998: Recommended Practice for Software Requirements Specifications.
- RTU Kota & Arya College 5th Semester PBL Guidelines (Academic Year 2026–27).
- Spring AI Reference Documentation (Spring Boot 3.3.x).
- PostgreSQL `pgvector` Extension Documentation.

---

2. OVERALL DESCRIPTION

2.1 Product Perspective

DocuQuery AI is an independent, modular system bridging local desktop user workstations with cloud-hosted transactional and vector databases. The architecture decouples the resource-intensive document chunking and vector similarity retrieval into a robust Spring Boot microservice, exposing Server-Sent Events (SSE) to an Electron-wrapped React 18 client.

2.2 Product Functions

1. User registration, authentication, and session handling via stateless JWT tokens.
2. Drag-and-drop document ingestion supporting PDF, DOCX, TXT, and Markdown files up to 25MB.
3. Recursive sliding-window chunking (500 tokens / 50-token overlap) preserving paragraph integrity.
4. Vector embedding generation and storage in Neon PostgreSQL with HNSW cosine distance indexing.
5. Contextual prompt assembly injecting retrieved source passages with strict anti-hallucination guardrails.
6. Real-time token streaming to the desktop client.

7. Clickable citation badges opening the exact source document and page excerpt.
8. Admin telemetry dashboard rendering RAG Triad evaluation metrics (Faithfulness, Relevance, Groundedness).

2.3 User Classes and Characteristics

- **Student / General User**: Queries the repository, inspects source citations, and reviews chat history.
- **Researcher / Faculty**: Uploads academic papers, creates topic workspaces, and reviews similarity metrics.
- **Administrator**: Re-indexes vector tables, purges corrupt files, and monitors latency and token costs.

2.4 Operating Environment

- **Server**: Java 21 LTS Runtime, Spring Boot 3.3.4, Neon PostgreSQL 16 with `pgvector` 0.7+.
- **Client**: Windows 10/11, macOS, Linux (Electron 30 runtime, minimum 4GB RAM, 500MB free disk space).

2.5 Design Constraints

- Primary backend programming language must strictly be **Java** (PBL requirement).
- Vector search latency must remain under 150ms for sub-second first-token response times.
- All cloud database transmissions must enforce TLS 1.3 encryption.

---

3. FUNCTIONAL REQUIREMENTS (FR)

Req ID	Module	Title	Description	Priority
FR-01	Ingestion	File Upload	System shall accept PDF, DOCX, TXT, and MD files up to 25MB via multipart POST.	High
FR-02	Ingestion	Text Parsing	Apache Tika shall extract raw text and metadata (page numbers, author, creation timestamp).	High
FR-03	Ingestion	Recursive Chunking	Text shall be partitioned into 500-token chunks with 50-token sliding overlap.	High
FR-04	Ingestion	Async Status	Upload API shall return HTTP 202 Accepted with a job UUID to track pipeline progression.	Medium
FR-05	Database	Vector Storage	System shall persist 1536-dimensional float vectors in `pgvector` columns.	High
FR-06	Database	HNSW Indexing	Vector table shall maintain an HNSW index using `vector_cosine_ops` ($m=16, ef=64$).	High
FR-07	Security	JWT Auth	System shall issue HMAC-SHA256 tokens valid for 24 hours on `/api/auth/login`.	High
FR-08	Security	RBAC	Endpoints shall restrict admin actions (`ROLE_ADMIN`) from student users.	High
FR-09	Security	Rate Limiting	Bucket4j shall reject client requests exceeding 30 req/min with HTTP 429.	Medium
FR-10	RAG Engine	Prompt Assembly	System prompt shall command model to answer exclusively from retrieved context passages.	High
FR-11	RAG Engine	Token Streaming	Responses shall stream token-by-token using Spring WebFlux Server-Sent Events (SSE).	High
FR-12	RAG Engine	Dynamic Citations	Each generated claim shall link to verified source document chunks `[Doc: X, Page: Y]`.	High
FR-13	Frontend	Desktop Shell	Electron shall provide native window controls and OS-level file drag-and-drop.	Medium
FR-14	Frontend	Markdown Render	Chat canvas shall format markdown, code syntax highlighting, and KaTeX math.	Medium
FR-15	Evaluation	Telemetry Metrics	System shall compute and display Context Relevance, Faithfulness, and Retrieval Latency.	Medium

---

4. EXTERNAL INTERFACE REQUIREMENTS

4.1 User Interfaces

- Modern, responsive dark-mode interface built with Tailwind CSS.
- Split-pane workspace: Navigation sidebar, central chat canvas, citation inspector drawer, and admin dashboard.

4.2 Software Interfaces

- **Neon PostgreSQL Database API**: JDBC connection string over TLS with HikariCP connection pooling.

- **LLM Embedding Interface**: HTTP REST client connecting to vector generation models (`text-embedding-3-small`).

---

5. NON-FUNCTIONAL REQUIREMENTS (NFR)

- **NFR-01 (Performance)**: Neon vector cosine search shall return Top-4 chunks within 120ms. Total first-token arrival shall remain under 1,000ms.
- **NFR-02 (Reliability)**: The system shall maintain 99.5% uptime during evaluation testing with zero JVM memory leak crashes.
- **NFR-03 (Security)**: User passwords must be salted and hashed with BCrypt (strength 12).
- **NFR-04 (Portability)**: The Electron desktop build shall execute identically across Windows 10/11 x64 systems without requiring manual JDK configuration.

---

6. VERIFICATION MATRIX

Every requirement in this specification is mapped to automated JUnit 5 tests, integration tests, and manual acceptance checklists satisfying Arya College's PBL evaluation criteria.